# U-Boot `uEnv.txt` : Seri Porttan Yükleme ve Rootfs'i eMMC'ye Taşıma (BeagleBone Black)

Bu dokümanda üç şey yapacağız:

1. `uEnv.txt` dosyasının **satır satır ne yaptığını** anlayacağız.
2. Bu dosyayı SD karta kopyalamadan, **seri port (UART) üzerinden** doğrudan U-Boot'a göndereceğiz.
3. Challenge: Kernel ve DTB'yi **SD karttan** yükleyip rootfs'i **eMMC'den** bağlayacağız.

> **Not**
>
> Bu dokümandaki terminal görüntüleri, adımları göstermek için hazırlanmış **temsili** çıktılardır. Senin kartında U-Boot sürümü, dosya boyutları ve bölüm boyutları farklı görünebilir.
>
> SD karttan boot'un temelleri için: booting.md · eMMC'ye kurulum için: eMMC-booting.md

---

## 1. Temel Kavramlar

| Kavram | Ne işe yarar? |
|---|---|
| `uEnv.txt` | U-Boot ortam değişkenlerini **metin olarak** tanımlayan dosya. Her satır `isim=değer` biçimindedir. U-Boot'un bu dosyayı açılışta otomatik okuyup okumaması, U-Boot'un yapılandırmasına bağlıdır (BBB'nin varsayılan U-Boot'u SD karttaki `uEnv.txt`'yi arar). |
| **Ortam değişkeni (environment)** | U-Boot'un ayarları ve makroları. `printenv` ile görülür, `setenv` ile değiştirilir, `saveenv` ile kalıcı kaydedilir. |
| **Minicom** | Bilgisayardan seri port (UART) üzerinden karta bağlanmayı sağlayan terminal programı. Kartın konsolu buraya düşer. |
| `loady` | U-Boot'ta **YMODEM** protokolüyle seri porttan dosya alıp RAM'e yazan komut. |
| `env import -t <adres> <boyut>` | RAM'deki `isim=değer` metnini okuyup U-Boot'un **aktif** ortamına ekler. `-t` "düz metin" demektir, `<boyut>` metnin bayt cinsinden uzunluğudur. |

Bütün akış şöyle:

```
Bilgisayar (uEnv.txt) ──YMODEM / UART──► RAM 0x82000000 ──env import──► U-Boot ortamı ──boot──► Linux
```

## 2. `uEnv.txt` Satır Satır

Kullanacağımız dosya bu:

```
mypcip=setenv serverip 192.168.1.2
ipaddr=192.168.27.1
bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw
bootcmd=echo "**********Booting from memory**********";load mmc 0:1 0x82000000
uImage;load mmc 0:1 0x88000000 am335x-boneblack.dtb;bootm 0x82000000 - 0x88000000
```

### `mypcip` ve `ipaddr` — ağ ayarları

| Satır | Anlamı |
|---|---|
| `mypcip=setenv serverip 192.168.1.2` | Bir **makro**. Kendi başına hiçbir şey yapmaz; `run mypcip` yazınca içindeki komut çalışır ve `serverip` (TFTP/NFS sunucusunun, yani bilgisayarın IP'si) `192.168.1.2` olur. |
| `ipaddr=192.168.27.1` | Kartın kendi statik IP adresi. |

> **Uyarı**
> Bu örnekte kart `192.168.27.x`, sunucu `192.168.1.x` ağında. Ağ geçidi (`gatewayip`) tanımlanmadığı sürece TFTP/NFS çalışmaz. Ağdan boot yapacaksan iki adresi **aynı alt ağda** seç. SD karttan boot için bu iki satır kullanılmaz.

### `bootargs` — kernel'e verilen parametreler

`bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw`

| Parça | Anlamı |
|---|---|
| `console=ttyO0,115200` | Linux mesajları UART0'a (J1 header) 115200 baud ile yazılsın. `ttyO0` eski OMAP kernel'lerindeki addır; yeni kernel'lerde `ttyS0` olur. |
| `root=/dev/mmcblk0p2` | Root filesystem: MMC aygıtı **0**, bölüm **2** (`p2`). |
| `rw` | Rootfs okuma-yazma modunda bağlansın. |

### `bootcmd` — otomatik açılış komutu

`bootcmd`, `;` ile ayrılmış komutlardan oluşur. U-Boot bunları sırayla çalıştırır:

| Sıra | Komut | Ne yapar? |
|---|---|---|
| 1 | `echo "****Booting from memory****"` | Ekrana mesaj basar (hangi yoldan açıldığını görmek için). |
| 2 | `load mmc 0:1 0x82000000 uImage` | `mmc 0:1` = SD kart, 1. bölüm (BOOT). Oradaki `uImage` dosyasını RAM'de `0x82000000` adresine yükler. |
| 3 | `load mmc 0:1 0x88000000 am335x-boneblack.dtb` | Aynı bölümdeki device tree dosyasını (DTB) `0x88000000` adresine yükler. |
| 4 | `bootm 0x82000000 - 0x88000000` | Kernel'i başlatır. Ortadaki `-` "initramfs yok" demektir, son adres DTB'nin adresidir. |

```
RAM (DDR, 0x80000000'dan başlar)
├── 0x82000000  →  uImage   (kernel)
└── 0x88000000  →  am335x-boneblack.dtb
```

> **Not**
> `mmc 0` ↔ `/dev/mmcblk0` eşleşmesi çoğu kurulumda doğrudur ama **garanti değildir**. U-Boot tarafında `mmc list`, Linux tarafında `lsblk` ile kontrol et (bkz. Bölüm 4).

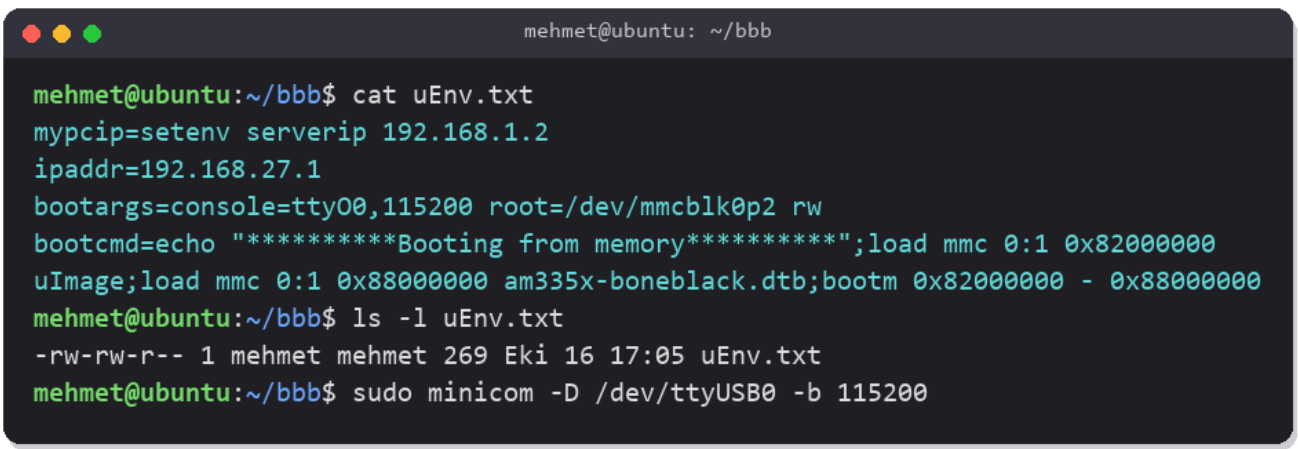
---

## 3. Uygulama: `uEnv.txt`'yi Seri Porttan Göndermek

Normalde `uEnv.txt` SD kartın BOOT bölümüne kopyalanır. Her değişiklikte SD kartı çıkar-tak yapmak yerine dosyayı **seri porttan** gönderip anında deneyebiliriz.

### Adım 1 — Bilgisayarda dosyayı hazırla ve Minicom'u aç

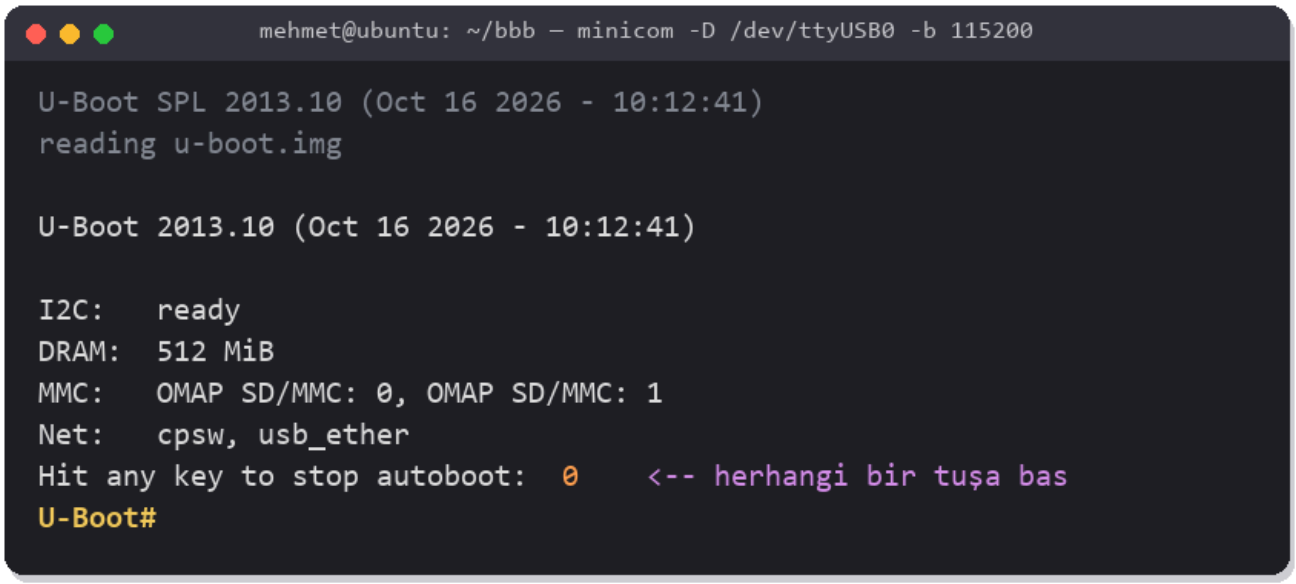
Dosyanın içeriğini ve **boyutunu** kontrol et, sonra Minicom ile karta bağlan:



> **İpucu**
> Windows'ta port adı `/dev/ttyUSB0` yerine `COM3` gibi olur. Minicom kurulumu için: minicom.md

### Adım 2 — U-Boot'u durdur

Kartı aç. `Hit any key to stop autoboot` yazısı çıkınca **bir tuşa bas**. `U-Boot#` komut satırı gelmeli:




### Adım 3 — `loady` ile dosyayı RAM'e al

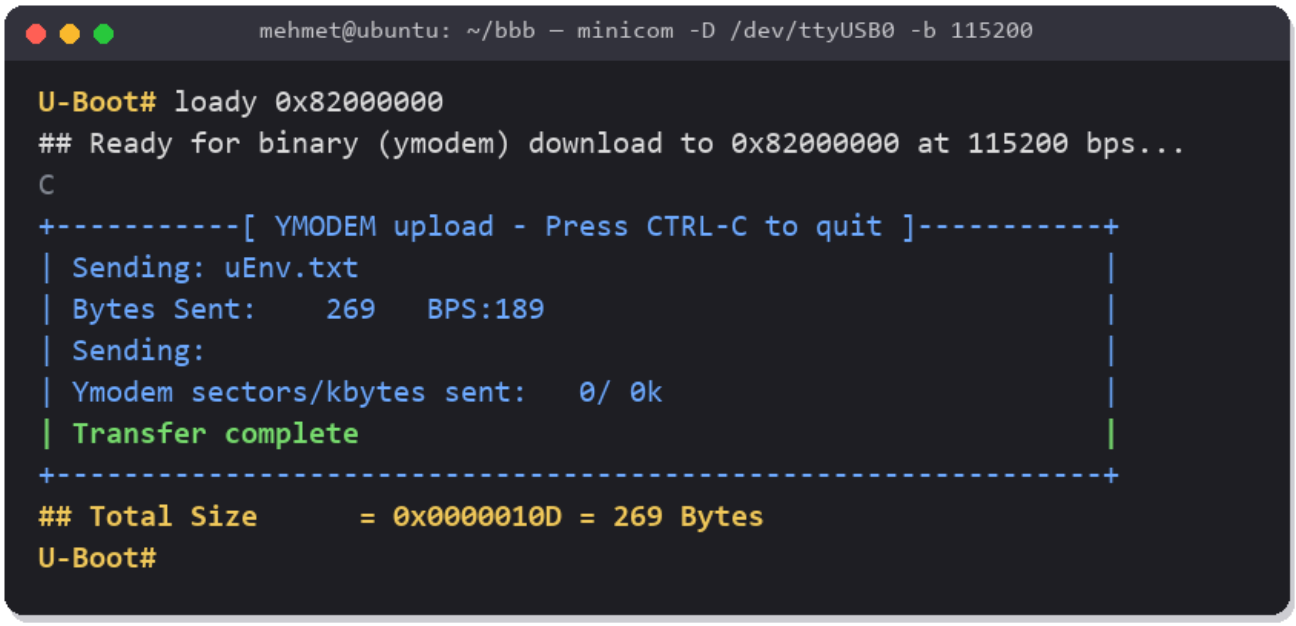
U-Boot'ta YMODEM alımını başlat:

```
U-Boot# loady 0x82000000
```

U-Boot `C` karakterleri basarak beklemeye başlar. Şimdi Minicom'da:

1. `Ctrl-A` → `S` (Send files)
2. Protokol olarak **ymodem** seç
3. `uEnv.txt` dosyasını seçip gönder

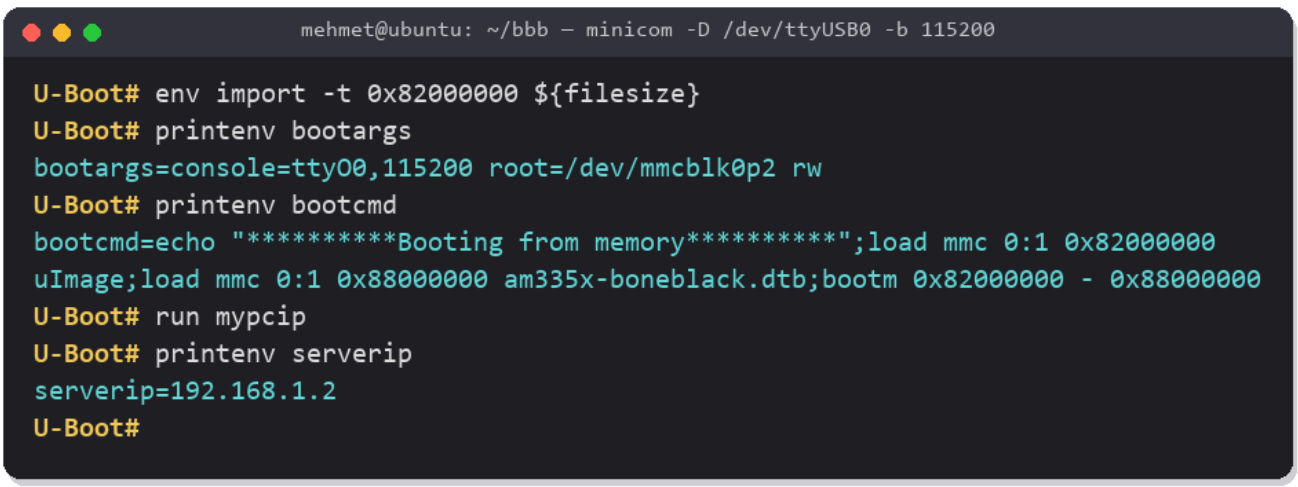
Transfer bitince U-Boot alınan dosyanın boyutunu yazar:




> **Not**
>
> `## Total Size = 0x0000010D = 269 Bytes` satırı önemli: Bu değer otomatik olarak **`filesize`** değişkenine yazılır. Bir sonraki adımda boyutu elle yazmak yerine `${filesize}` kullanabiliriz.

### Adım 4 — `env import` ile ortama aktar ve kontrol et

```
U-Boot# env import -t 0x82000000 ${filesize}
U-Boot# printenv bootargs
U-Boot# printenv bootcmd
```



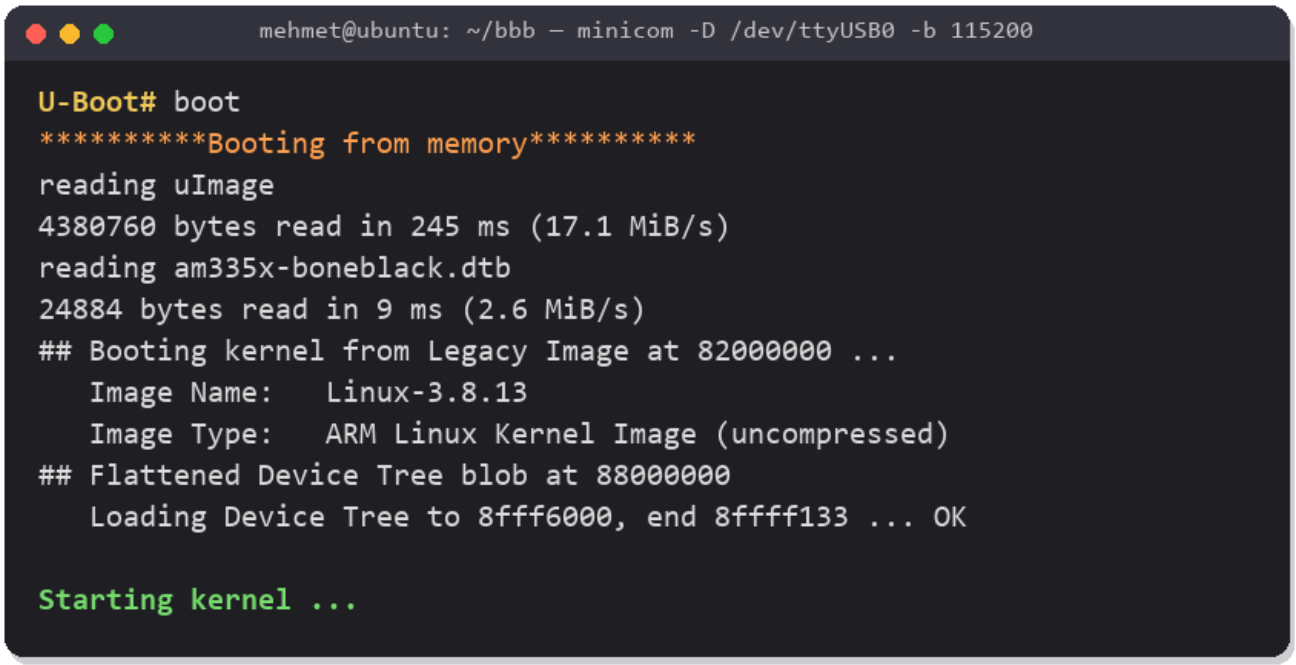

`printenv` çıktısı `uEnv.txt`'deki değerlerle aynıysa aktarım başarılıdır. `run mypcip` ile makronun da çalıştığını görebilirsin.

> **Uyarı**
>
> `${filesize}` yerine boyutu elle yazarsan **hex** olarak yaz ( `0x10d` veya `10d` ). U-Boot sayıları varsayılan olarak hex kabul eder; `269` yazarsan `0x269` = 617 bayt okur ve dosyanın sonundaki çöp veriyi de içe aktarmaya çalışır.

### Adım 5 — Boot et

```
U-Boot# boot
```

`boot` komutu `bootcmd`'yi çalıştırır. Ekrana bizim `echo` mesajımız gelmeli:



### Dikkat edilecekler

| Konu | Açıklama |
|---|---|
| **Adres çakışması** | `uEnv.txt` `0x82000000`'a yüklendi; `bootcmd` de kernel'i aynı adrese yüklüyor. Sorun değil, çünkü `env import` dosyayı önce okuyup bitiriyor, kernel **sonra** üzerine yazılıyor. Sıra ters olursa kernel bozulur. |
| **Geçicilik** | `env import` sadece RAM'deki aktif ortamı değiştirir. Kart yeniden açılınca değişiklikler kaybolur. |
| **Kalıcı yapmak** | Her şeyin doğru olduğundan emin olduktan sonra `saveenv` ile kaydedilebilir. Bu komut kalıcı U-Boot ortamının üzerine yazar; geri dönmek için `env default -a` + `saveenv` gerekir. |
| **Otomatik okumada `bootcmd`** | BBB'nin varsayılan U-Boot'u SD karttaki `uEnv.txt`'yi kendi `bootcmd`'si **çalışırken** okur ve `uenvcmd` değişkenini arar. Bu yüzden SD karttaki `uEnv.txt`'de `bootcmd` tanımlamak o açılışı etkilemez; orada `uenvcmd` kullanılır (booting.md'deki örnek gibi). Seri porttan yüklemede ise `boot` komutunu biz verdiğimiz için `bootcmd` çalışır. |

---

## 4. Challenge: Kernel ve DTB SD Karttan, Rootfs eMMC'den

### Görev

| Ne? | Nereden? |
|---|---|
| Kernel ( `uImage` ) | **MMC0** — microSD kart, 1. bölüm |
| Device tree ( `.dtb` ) | **MMC0** — microSD kart, 1. bölüm |
| Root filesystem | **MMC1** — dahili eMMC, 2. bölüm |

```
microSD (mmc 0)                        eMMC (mmc 1)
├── p1 BOOT   → uImage, .dtb  ──┐      ├── p1 BOOT
└── p2 ROOTFS                   │      └── p2 ROOTFS   ◄── root=/dev/mmcblk1p2
                                ▼
                  U-Boot RAM'e yükler → kernel başlar → rootfs'i eMMC'den bağlar
```

### Adım 1 — Aygıt numaralarını doğrula

Önce varsayımımızı kontrol edelim: `mmc 0` gerçekten SD kart, `mmc 1` gerçekten eMMC mi ve eMMC'de 2. bölüm var mı?

```
U-Boot# mmc list
U-Boot# mmc dev 1
U-Boot# mmc part
```

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

U-Boot# mmc list
OMAP SD/MMC: 0       <-- mmc 0 : microSD kart
OMAP SD/MMC: 1       <-- mmc 1 : dahili eMMC
U-Boot# mmc dev 1
mmc1(part 0) is current device
U-Boot# mmc part

Partition Map for MMC device 1  --   Partition Type: DOS

Part    Start Sector    Num Sectors     UUID            Type
  1     2048            196608          00000000-01     0c Boot
  2     198656          7266304         00000000-02     83   <-- rootfs
U-Boot#
```

eMMC'de `83` (Linux) tipinde 2. bölüm görünüyorsa rootfs için doğru yerdeyiz.

## Adım 2 — Sadece `bootargs`'ı değiştir

Kernel ve DTB hâlâ SD karttan yükleneceği için **`bootcmd` aynı kalır**. Değişen tek şey, kernel'e "rootfs nerede?" bilgisini veren `bootargs`:

| | Değer |
|---|---|
| **Eski** — rootfs SD kartta | `bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw` |
| **Yeni** — rootfs eMMC'de | `bootargs=console=ttyO0,115200 root=/dev/mmcblk1p2 rw` |

Bunu iki yolla yapabilirsin:

- **a)** `uEnv.txt`'deki satırı değiştirip Bölüm 3'teki gibi tekrar gönder.
- **b)** Doğrudan U-Boot'ta `setenv` ile değiştir:

```
U-Boot# setenv bootargs console=ttyO0,115200 root=/dev/mmcblk1p2 rw
U-Boot# boot
```

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

U-Boot# printenv bootargs
bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw
U-Boot# setenv bootargs console=ttyO0,115200 root=/dev/mmcblk1p2 rw
U-Boot# printenv bootargs
bootargs=console=ttyO0,115200 root=/dev/mmcblk1p2 rw
U-Boot# boot
**********Booting from memory**********
reading uImage
...
Starting kernel ...
```

### Adım 3 — Linux'ta sonucu doğrula

Kart açılınca rootfs'in gerçekten eMMC'den bağlandığını kontrol et:

```
cat /proc/cmdline   # kernel'e giden bootargs
lsblk               # hangi bölüm / olarak bağlı?
findmnt /           # rootfs'in kaynağı
```

```
root@beaglebone: ~ (seri konsol)

root@beaglebone:~# cat /proc/cmdline
console=ttyO0,115200 root=/dev/mmcblk1p2 rw
root@beaglebone:~# lsblk
NAME         MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
mmcblk0      179:0    0  7.4G  0 disk
|-mmcblk0p1  179:1    0   96M  0 part
`-mmcblk0p2  179:2    0  7.3G  0 part
mmcblk1      179:8    0  3.6G  0 disk
|-mmcblk1p1  179:9    0   96M  0 part
`-mmcblk1p2  179:10   0  3.5G  0 part /          <-- rootfs eMMC'de
root@beaglebone:~# findmnt /
TARGET SOURCE         FSTYPE OPTIONS
/      /dev/mmcblk1p2 ext4   rw,relatime
```

`/` bağlama noktası `mmcblk1p2`'nin yanında görünüyorsa challenge tamam.

## Sonuç

| Bileşen | Durum |
| --- | --- |
| **Kernel ve DTB** | microSD'nin 1. bölümünden RAM'e yüklenir. |
| **Rootfs** | Kernel açıldıktan sonra eMMC'nin 2. bölümünden bağlanır. |
| `bootcmd` | Değişmez; yükleme kaynağı hâlâ SD kart. |
| `bootargs` | `root=/dev/mmcblk1p2` olarak güncellenir. |

> **Önemli**
>
> Bunun çalışması için: - eMMC'nin 2. bölümünde gerçekten bir Linux rootfs olmalı. - Kernel'de MMC sürücüsü ve rootfs'in dosya sistemi (ör. ext4) desteği **built-in** olmalı (veya bir initramfs'ten yüklenmeli). - Linux'taki numaralandırma U-Boot'unkiyle aynı olmalı. Değilse kernel `VFS: Unable to mount root fs` hatasıyla durur; bu durumda `root=/dev/mmcblk0p2` dene veya `root=PARTUUID=...` kullan.

---

## 5. Hızlı Tekrar

| Komut / Değişken | Görevi |
|---|---|
| `uEnv.txt` | U-Boot ortam değişkenlerini metin olarak tanımlar |
| `loady <adres>` | YMODEM ile seri porttan RAM'e dosya alır, boyutu `filesize`'a yazar |
| `env import -t <adres> ${filesize}` | RAM'deki metin değişkenlerini aktif ortama aktarır |
| `printenv <isim>` | Değişkenin değerini gösterir |
| `setenv <isim> <değer>` | Değişkeni değiştirir (geçici) |
| `run <makro>` | Değişkendeki komutları çalıştırır (ör. `run mypcip`) |
| `boot` | `bootcmd`'yi çalıştırır |
| `bootcmd` | Otomatik açılışta çalışan komut dizisi |
| `bootargs` | Linux kernel'ine verilen açılış parametreleri |
| `root=/dev/mmcblk1p2` | Rootfs'in eMMC'nin 2. bölümünde olduğunu söyler |
| `mmc list` / `mmc part` | U-Boot'un gördüğü MMC aygıtlarını ve bölümleri listeler |
| `saveenv` | Aktif ortamı kalıcı olarak kaydeder (dikkatli kullan) |